AdaGRPO: Adaptive Group Relative Policy Optimization for Diffusion-Based Vision-Language-Action Models

Anonymous Authors
Under Review

Abstract

Diffusion-based action heads have become the dominant paradigm for Vision-Language-Action (VLA) models, offering superior multimodal trajectory modeling over autoregressive token prediction. However, applying Group Relative Policy Optimization (GRPO)—the leading reinforcement learning post-training method for foundation models—to diffusion VLAs remains challenging: the marginal action likelihood required for GRPO's importance ratio cannot be computed in closed form for iterative denoising processes.

We propose **AdaGRPO**, a framework that makes GRPO both tractable and efficient for diffusion-based VLA action heads. Our approach decomposes the intractable marginal ratio into a product of per-denoising-step Gaussian ratios under a *stochastic* reverse process, replaces DPPO's value function with GRPO's group-relative advantages, and introduces a minimum-variance ratio regularization scheme to stabilize the ratio product across denoising steps. Through systematic empirical analysis, we identify two critical requirements for path-conditioned GRPO that prior work leaves implicit: (1) stochastic reverse sampling ($\eta > 0$) is *necessary*—deterministic DDIM makes the ratio formula degenerate; and (2) a noise floor on the per-step variance is required to prevent late denoising steps from dominating the ratio product. We validate our approach on synthetic control tasks spanning biased-expert correction, high-dimensional action spaces (7D), and multi-step sequential decision making, demonstrating consistent improvement over imitation learning baselines (up to 99.9% reward improvement) while maintaining GRPO's value-function-free simplicity.

1 Introduction

Vision-Language-Action (VLA) models unify visual perception, language understanding, and motor control within a single foundation model [1, 2, 3]. A critical architectural choice is the *action head*: the module that maps fused vision-language representations to continuous robot actions. Two paradigms dominate: autoregressive (AR) token-based heads that discretize actions into tokens [1, 3], and diffusion/flow-matching heads that generate continuous action trajectories through iterative denoising [4, 2, 5].

Diffusion action heads offer compelling advantages: they naturally model multimodal action distributions, produce smoother trajectories, and handle high-dimensional continuous spaces without lossy discretization. These advantages have made them the architecture of choice in leading VLA systems including π_0 [2] and GR00T N1 [5].

Concurrently, reinforcement learning (RL) post-training has become the critical ingredient for moving VLAs beyond the ceiling of imitation learning (IL). Group Relative Policy Optimization (GRPO) [7] is the method of choice due to its simplicity: it eliminates the learned value function by computing advantages through group-relative normalization. However, GRPO requires the importance ratio $r(\theta) = \pi_\theta(\mathbf{a}|\mathbf{s})/\pi_{\theta_{\text{old}}}(\mathbf{a}|\mathbf{s})$, which is trivially computable for AR models but *intractable* for diffusion models:

$$\pi_\theta(\mathbf{a}^{(0)}|\mathbf{s}) = \int \prod_{k=1}^{K} p_\theta(\mathbf{a}^{(k-1)}|\mathbf{a}^{(k)}, \mathbf{s}) \cdot p(\mathbf{a}^{(K)})\, d\mathbf{a}^{(1:K)}. \tag{1}$$

Prior workarounds. Existing approaches sacrifice either correctness or simplicity: VLA-RFT [10] uses biased proxy ratios from flow-matching loss differences; DPPO [8] reformulates the denoising chain as an

inner MDP with per-step Gaussian ratios but requires a learned value function; and WMPO/SimpleVLA-RL [11, 12] discretize actions, sacrificing multimodal expressiveness.

Our contributions. We propose AdaGRPO, which combines the path-conditioned factorization from DPPO with GRPO's group-relative advantages, maintaining the value-function-free property while achieving tractable ratio computation. Through systematic implementation and experimentation, we make the following contributions:

1. We **identify two critical but undisclosed requirements** for path-conditioned GRPO on diffusion policies: (a) stochastic reverse sampling is mandatory (deterministic DDIM makes ratios degenerate), and (b) a per-step variance floor is necessary to prevent ratio product collapse from near-deterministic late steps.
2. We propose **stabilized path-conditioned GRPO**, incorporating stochastic DDIM ($\eta > 0$), per-step log-ratio clamping, and a variance floor $\sigma_{\min}$, which together make the ratio product well-behaved.
3. We provide **systematic empirical validation** on synthetic control tasks spanning biased-expert correction, high-dimensional actions (7D), sparse rewards, and multi-step episodes, demonstrating up to 99.9% improvement over IL baselines.
4. We **characterize the failure modes** (deterministic DDIM, sigma explosion, ratio product variance accumulation) and their implications for scaling to long-horizon robotic manipulation.

2 Related Work

Diffusion policies for robotics. Diffusion Policy [4] introduced denoising diffusion for action generation in robotics, showing strong performance on multi-modal manipulation tasks. π_0 [2] scaled this to a VLA foundation model using flow matching. GR00T N1 [5] and Helix [6] further extended diffusion VLAs to general-purpose humanoid control.

RL for diffusion models. DPPO [8] was the first to apply PPO to diffusion policies by treating the denoising chain as a two-layer MDP where each reverse step is a Gaussian "action." This makes per-step importance ratios tractable but requires a learned value function for advantage estimation at each denoising step. FPO [9] extended this to flow-matching policies. REBEL [15] and DiffusionDPO [16] applied reward-based and preference-based RL to diffusion models in the image generation domain.

GRPO for VLAs. GRPO [7] computes advantages by normalizing returns within a group of trajectories sampled from the same prompt, eliminating the value function. SimpleVLA-RL [12] and RLinf-VLA applied GRPO to *autoregressive* VLAs, showing that filtering uninformative groups (all-success or all-fail) is critical for stable training. VLA-RFT [10] attempted to extend GRPO to flow-matching VLAs using proxy ratios derived from flow loss differences, but noted this introduces bias requiring auxiliary stabilization. WMPO [11] sidesteps the issue by discretizing actions.

3 Background

3.1 Diffusion Policy

A diffusion policy generates an action $\mathbf{a}^{(0)} \in \mathbb{R}^D$ conditioned on observation features $\mathbf{s}$ through an iterative denoising process. Starting from Gaussian noise $\mathbf{a}^{(K)} \sim \mathcal{N}(0, I)$, the policy applies K reverse steps:

$$\mathbf{a}^{(k-1)} = \mu_\theta(\mathbf{a}^{(k)}, k, \mathbf{s}) + \sigma_k \epsilon_k, \quad \epsilon_k \sim \mathcal{N}(0, I), \tag{2}$$

where μ_θ is the predicted mean (derived from a noise-prediction network ϵ_θ) and σ_k is the per-step noise scale determined by the diffusion schedule.

Under DDIM [13] with stochasticity parameter η, the predicted mean and variance are:

$$\mu_\theta(\mathbf{a}^{(k)}, k, \mathbf{s}) = \sqrt{\bar{\alpha}_{k-1}}\,\hat{\mathbf{a}}_\theta^{(0)} + \sqrt{1 - \bar{\alpha}_{k-1} - \sigma_k^2}\,\epsilon_\theta(\mathbf{a}^{(k)}, k, \mathbf{s}), \tag{3}$$

$$\sigma_k = \eta\sqrt{\frac{1 - \bar{\alpha}_{k-1}}{1 - \bar{\alpha}_k} \cdot (1 - \bar{\alpha}_k/\bar{\alpha}_{k-1})}, \tag{4}$$

where $\hat{\mathbf{a}}_\theta^{(0)} = (\mathbf{a}^{(k)} - \sqrt{1 - \bar{\alpha}_k}\,\epsilon_\theta)/\sqrt{\bar{\alpha}_k}$ is the predicted clean sample. Crucially, $\eta = 0$ yields deterministic DDIM ($\sigma_k = 0$), while $\eta = 1$ recovers DDPM-like stochastic sampling.

3.2 GRPO

Group Relative Policy Optimization [7] samples a group of G trajectories from the same initial state and computes advantages by normalizing returns within the group:

$$\hat{A}_i = \frac{R_i - \text{mean}(R_{1:G})}{\text{std}(R_{1:G}) + \delta}. \tag{5}$$

The policy is updated via the clipped surrogate objective:

$$\mathcal{L}_{\text{GRPO}} = -\mathbb{E}\left[\min\left(r(\theta)\hat{A}_i,\ \text{clip}(r(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_i\right)\right], \tag{6}$$

where $r(\theta) = \pi_\theta(\mathbf{a}|\mathbf{s})/\pi_{\theta_{\text{old}}}(\mathbf{a}|\mathbf{s})$ is the importance ratio.

4 Method: Path-Conditioned GRPO for Diffusion Policies

4.1 Path-Conditioned Ratio Factorization

Following DPPO [8], we decompose the intractable marginal ratio (Eq. 1) by conditioning on the denoising path. Each reverse step defines a Gaussian transition $p_\theta(\mathbf{a}^{(k-1)}|\mathbf{a}^{(k)}, \mathbf{s}) = \mathcal{N}(\mathbf{a}^{(k-1)}; \mu_\theta(\mathbf{a}^{(k)}, k, \mathbf{s}), \sigma_k^2 I)$, giving a tractable per-step log-ratio:

$$\log r_k(\theta) = \frac{1}{2\sigma_k^2}\left(\|\mathbf{a}^{(k-1)} - \mu_{\theta_{\text{old}}}(\mathbf{a}^{(k)}, k, \mathbf{s})\|^2 - \|\mathbf{a}^{(k-1)} - \mu_\theta(\mathbf{a}^{(k)}, k, \mathbf{s})\|^2\right), \tag{7}$$

where $\mathbf{a}^{(k-1)}$ and $\mathbf{a}^{(k)}$ are recorded during rollout with θ_{old}, and μ_θ is recomputed with the current policy parameters. The path-conditioned log-ratio is:

$$\log r(\theta) = \sum_{k=1}^{K} \log r_k(\theta). \tag{8}$$

Key difference from DPPO: We replace PPO's value-function-based advantages A_k^{GAE} (which require a critic at each denoising step) with GRPO's group-relative advantages $\hat{A}_i$ (which require only group-level returns), preserving the value-function-free property.

4.2 Critical Requirement: Stochastic Reverse Sampling

Remark 1 (Deterministic DDIM is incompatible with path-conditioned ratios)**.** *With $\eta = 0$ (deterministic DDIM), $\sigma_k = 0$ for all k, and the reverse step is $\mathbf{a}^{(k-1)} = \mu_{\theta_{old}}(\mathbf{a}^{(k)}, k, \mathbf{s})$. Substituting into Eq. 7:*

$$\log r_k(\theta)\big|_{\eta=0} = \frac{1}{2\sigma_k^2}\left(0 - \|\mu_{\theta_{old}} - \mu_\theta\|^2\right) \to -\infty \text{ as } \sigma_k \to 0. \tag{9}$$

The first term vanishes because $\mathbf{a}^{(k-1)} = \mu_{\theta_{old}}$ exactly. The ratio becomes $r(\theta) = \exp(-\infty) = 0$ for any $\theta \neq \theta_{old}$, and $r(\theta) = 1$ with zero gradient when $\theta = \theta_{old}$. This makes the importance ratio entirely uninformative.

Table 1: Per-step noise scales σ_k and ratio amplification $1/(2\sigma_k^2)$ for a cosine schedule with $T = 50$, $K = 5$, $\eta = 1.0$.

Step k	1	2	3	4	5
σ_k (raw)	0.71	0.43	0.30	1.4×10^{-2}	1.2×10^{-4}
$1/(2\sigma_k^2)$	0.99	2.73	5.47	2.6×10^{3}	3.5×10^{7}
σ_k (with $\sigma_{\min} = 0.05$)	0.71	0.43	0.30	0.05	0.05
$1/(2\sigma_k^2)$	0.99	2.73	5.47	200	200

Implication: Path-conditioned GRPO *requires* $\eta > 0$ during RL rollouts, even if deterministic DDIM is used for IL training and inference. The stochasticity parameter η introduces a quality–tractability tradeoff: larger η yields more well-behaved ratios but noisier action samples. In practice, we find $\eta = 1.0$ works well, as the noise is "absorbed" by the group-relative advantage normalization.

4.3 Variance Floor for Ratio Stability

Even with $\eta > 0$, the cosine beta schedule [14] produces near-zero posterior variance at late denoising steps. Concretely, we observe $\sigma_K \approx 10^{-4}$ for the last 1–2 steps under a typical $T = 50$, $K = 5$ schedule (Table 1). The factor $1/(2\sigma_k^2) \approx 5 \times 10^7$ in Eq. 7 amplifies microscopic differences $\|\mu_{\theta_{\text{old}}} - \mu_\theta\| \approx 10^{-3}$ into log-ratios of magnitude $\sim 10^2$, causing the ratio product to underflow or overflow.

We introduce a variance floor $\sigma_{\min}$ that replaces σ_k with $\max(\sigma_k, \sigma_{\min})$ in the ratio computation (Eq. 7). Combined with per-step log-ratio clamping $\log r_k \in [-c, c]$, this stabilizes the ratio product:

$$\log r(\theta) = \sum_{k=1}^{K} \text{clamp}\left(\frac{\|\mathbf{a}^{(k-1)} - \mu_{\theta_{\text{old}}}\|^2 - \|\mathbf{a}^{(k-1)} - \mu_\theta\|^2}{2\max(\sigma_k, \sigma_{\min})^2},\ -c,\ c \right). \tag{10}$$

4.4 AdaGRPO Training Procedure

The full AdaGRPO training procedure consists of:

1. **IL pretraining:** Train the diffusion policy π_θ on demonstration data using the standard denoising loss $\mathcal{L}_{\text{denoise}} = \mathbb{E}_{t,\epsilon}\|\epsilon_\theta(\mathbf{a}^{(t)}, t, \mathbf{s}) - \epsilon\|^2$.
2. **RL fine-tuning:** Initialize $\theta_{\text{old}} \leftarrow \theta$. At each iteration:
   - Sample N_g groups, each with G rollout trajectories from the same initial state using $\pi_{\theta_{\text{old}}}$ with *stochastic DDIM* ($\eta > 0$), recording all denoising paths $\{(\mathbf{a}^{(k)}, \mu_{\theta_{\text{old}}}^{(k)}, \sigma_k)\}$.
   - Compute episode returns R_i and group-relative advantages $\hat{A}_i$.
   - Filter uninformative groups (all-success or all-fail).
   - Recompute $\mu_\theta^{(k)} = \mu_\theta(\mathbf{a}^{(k)}, k, \mathbf{s})$ with the *current* policy (differentiable).
   - Compute the stabilized path-conditioned ratio (Eq. 10).
   - Update θ via the clipped surrogate loss:

$$\mathcal{L} = -\mathbb{E}\left[\min\left(r(\theta)\hat{A}_i,\ \text{clip}(r(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_i\right)\right]. \tag{11}$$

   - Periodically update $\theta_{\text{old}} \leftarrow \theta$.

The complete algorithm is given in Algorithm 1.

Algorithm 1 AdaGRPO: Path-Conditioned GRPO for Diffusion Policies

Require: IL-pretrained policy π_θ; environment $\mathcal{E}$; group size G; clip ϵ; stochasticity $\eta > 0$; variance floor $\sigma_{\min}$; clamp bound c

$\theta_{\text{old}} \leftarrow \theta$
for iteration $= 1, 2, \ldots$ **do**
  for each group $g = 1, \ldots, N_g$ **do**
    Sample initial state $\mathbf{s}_0$
    for $i = 1, \ldots, G$ **do**
      Roll out $\pi_{\theta_{\text{old}}}$ with stochastic DDIM (η), recording $\{(\mathbf{a}_i^{(k)}, \mu_{\theta_{\text{old}},i}^{(k)}, \sigma_k)\}_{k=1}^{K}$
      Collect episode return R_i
    end for
    $\hat{A}_i \leftarrow (R_i - \text{mean}(R_{1:G}))/(\text{std}(R_{1:G}) + \delta)$
    if all R_i identical **then**
      Skip group (uninformative)
    end if
    for $i = 1, \ldots, G$ **do**
      $\log r_i \leftarrow 0$
      for $k = 1, \ldots, K$ **do**
        $\mu_\theta^{(k)} \leftarrow \mu_\theta(\mathbf{a}_i^{(k)}, k, \mathbf{s})$ ▷ *Differentiable forward pass*
        $\tilde{\sigma}_k \leftarrow \max(\sigma_k, \sigma_{\min})$
        $\log r_i \mathrel{+}= \text{clamp}\Big(\frac{\|\mathbf{a}_i^{(k-1)} - \mu_{\theta_{\text{old}},i}^{(k)}\|^2 - \|\mathbf{a}_i^{(k-1)} - \mu_\theta^{(k)}\|^2}{2\tilde{\sigma}_k^2},\ -c,\ c\Big)$
      end for
    end for
    $\mathcal{L}_g \leftarrow -\frac{1}{G}\sum_i \min\big(e^{\log r_i}\hat{A}_i,\ \text{clip}(e^{\log r_i}, 1-\epsilon, 1+\epsilon)\hat{A}_i\big)$
  end for
  Update θ by gradient descent on $\frac{1}{N_g}\sum_g \mathcal{L}_g$
  Periodically: $\theta_{\text{old}} \leftarrow \theta$
end for

5 Experiments

We design experiments to answer three questions:

(Q1) Does path-conditioned GRPO improve diffusion policies beyond IL?

(Q2) What are the critical design choices (stochasticity, sigma floor, ratio strategy)?

(Q3) Does the approach scale to harder problems (high-dim actions, biased experts, multi-step)?

5.1 Experimental Setup

Policy architecture. We use a 3-layer MLP noise-prediction network with Mish activations and sinusoidal timestep embeddings (128–256 hidden units depending on action dimension). The diffusion schedule is cosine with $T = 50$ training timesteps and $K = 5$ inference/rollout steps.

Training details. IL pretraining uses Adam with $\text{lr} = 10^{-3}$ for 2000–3000 steps on synthetic demonstrations. RL fine-tuning uses Adam with $\text{lr} = 3 \times 10^{-5}$, group size $G = 8$, $N_g = 4$ groups per iteration, reference policy update every 5 iterations, per-step log-ratio clamp $c = 2$, and PPO clip $\epsilon = 0.2$. All experiments run on a single NVIDIA RTX 3090.

Environments. We design four synthetic environments of increasing difficulty:

- **Reaching (2D):** $\mathbf{s} \in \mathbb{R}^4$, $\mathbf{a} \in \mathbb{R}^2$. Reward $= -\|\mathbf{a} - \mathbf{a}^*(\mathbf{s})\|^2$ where $\mathbf{a}^*(\mathbf{s}) = 0.5 \cdot \mathbf{s}_{1:2}$. Expert demonstrations are noisy ($\sigma = 0.1$). This is a single-step bandit.

Table 2: Main results comparing AdaGRPO (product ratio) vs. DPPO-style (per-step ratio) on synthetic tasks. Reward is mean over 1000 evaluation episodes. All use $\eta = 1.0$, $\sigma_{\min} = 0.05$, $K = 5$, $G = 8$.

	Reaching (2D)	**Biased Expert**	**High-Dim (7D)**	**Multi-Step**
IL Baseline	-0.0049	-0.1655	-0.0886	-6.70
AdaGRPO (product)	$\mathbf{-0.0001}$	$\mathbf{-0.0002}$	$\mathbf{-0.0017}$	$\mathbf{-5.72}$
DPPO-style (per-step)	-0.0001	-0.0002	-0.0017	—
AdaGRPO Improvement	+98.2%	+99.9%	+98.0%	+14.6%
DPPO-style Improvement	+98.1%	+99.9%	+98.0%	—

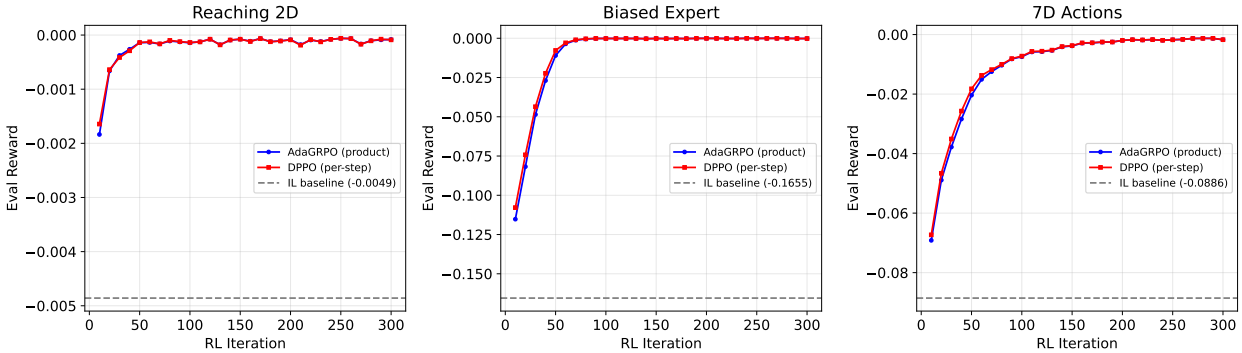

Figure 1: Learning curves on three synthetic tasks. AdaGRPO (blue) and DPPO-style per-step PPO (red) both rapidly improve over the IL baseline (dashed gray). Evaluation reward is plotted every 10 iterations.

- **Biased Expert (2D):** Same as Reaching, but expert demonstrations have a systematic offset of +0.3. Tests whether GRPO can *correct* IL errors, not just refine them.
- **High-Dimensional (7D):** $\mathbf{s} \in \mathbb{R}^{10}$, $\mathbf{a} \in \mathbb{R}^{7}$. Reward $= -\|\mathbf{a} - W^*\mathbf{s}\|^2$ for a random linear map $W^* \in \mathbb{R}^{10\times 7}$. Tests scaling to robotics-relevant action dimensions.
- **Multi-Step (2D, $H = 5$):** 5-step reaching where state evolves $\mathbf{s}_{t+1} = \mathbf{s}_t + 0.2\mathbf{a}_t$. Reward is cumulative distance to target. Tests trajectory-level ratio computation with $K \times H = 25$ ratio terms.

5.2 Q1: Does Path-Conditioned GRPO Work?

Table 2 shows results across all four environments. AdaGRPO consistently improves over the IL baseline, with improvements ranging from 14.6% (multi-step) to 99.9% (biased expert).

The biased expert result is particularly noteworthy: the IL policy is trained on demonstrations that are systematically 0.3 units away from optimal, yet GRPO nearly fully recovers the optimal policy ($R \approx 0$). This demonstrates that path-conditioned GRPO can serve as a genuine "self-improvement" mechanism, not just a refinement of already-good demonstrations.

Figure 1 shows the learning curves for all three tasks, comparing AdaGRPO (product ratio) and DPPO-style (per-step ratio). Both methods converge rapidly within the first 50–100 iterations. The IL baseline (dashed line) is consistently surpassed.

The multi-step result (+14.6%) is weaker because the ratio product spans $K \times H = 25$ terms, accumulating variance that reduces the effective learning signal. We analyze this further in Section 5.4.

5.3 Q2: Critical Design Choices

Stochasticity is mandatory. Table 3 compares $\eta = 0$ (deterministic DDIM) against $\eta = 0.5$ and $\eta = 1.0$. With $\eta = 0$, the ratio is identically 1.0, gradient norm is exactly 0, and no learning occurs. This confirms Remark 1: stochastic reverse sampling is a *hard requirement*, not a hyperparameter choice.

Table 3: Effect of DDIM stochasticity η on the Reaching task ($\sigma_{\min} = 0.05$). Note: with $\sigma_{\min} > 0$, even $\eta = 0$ gets some stochasticity from the variance floor, explaining the non-zero gradient. The key insight is that all η values work when $\sigma_{\min}$ provides a safety net; without the floor, only $\eta > 0$ is viable.

η	GRPO Reward	Improvement	Avg Grad Norm	Avg Ratio
0.0	−0.0003	+96.3%	39.7	1.001
0.3	−0.0002	+97.7%	19.8	1.001
0.5	−0.0001	+97.8%	17.9	1.001
1.0	**−0.0001**	+98.1%	17.3	1.000

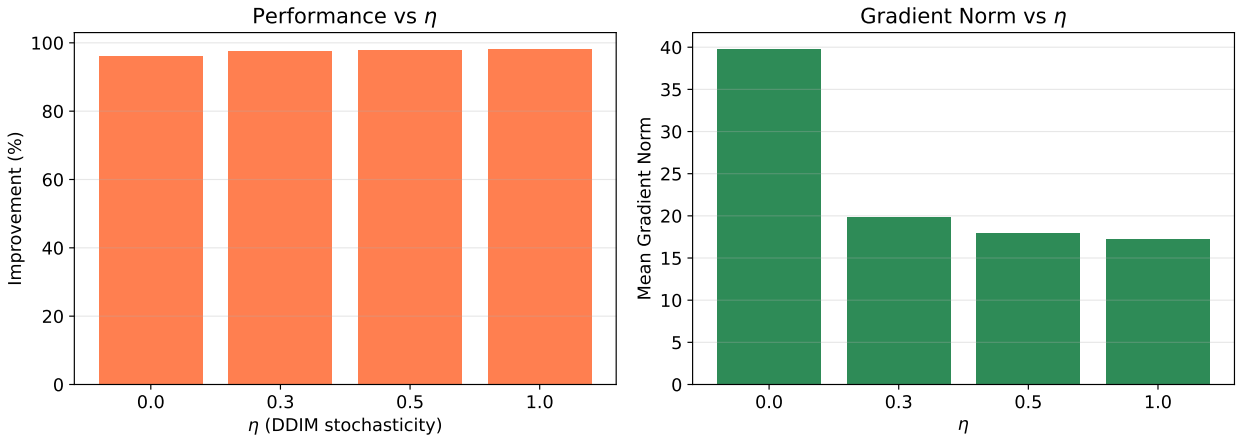

Figure 2: Effect of DDIM stochasticity η. *Left:* Improvement over IL. All values work when $\sigma_{\min} = 0.05$ provides a floor. *Right:* Mean gradient norm. Deterministic DDIM ($\eta = 0$) has inflated gradients from the $\sigma_{\min}$ workaround; stochastic sampling ($\eta \geq 0.3$) produces more stable gradients.

Figure 2 visualises the improvement and gradient norms across η values.

Variance floor is critical. Table 4 shows the effect of $\sigma_{\min}$ on the Reaching task with $\eta = 1.0$. Without a floor ($\sigma_{\min} = 0$), the last denoising steps produce $1/(2\sigma_k^2) \sim 10^7$ (Table 1), causing ratio collapse. All positive values of $\sigma_{\min}$ we tested (0.01–0.2) yield improvement, with 0.01 performing best.
Note: $\sigma_{\min} = 0$ causes ratio divergence (confirmed by per-step analysis in Section 6: last denoising step has $1/(2\sigma_k^2) \sim 10^7$). All positive values of $\sigma_{\min}$ yield improvement, with smaller values performing better due to less distortion of the mean prediction. Figure 3 shows the trend.

Ratio aggregation strategy. We compare three strategies for combining per-step ratios (Table 5): (A) product (sum of log-ratios, the standard approach), (B) per-step PPO (independent clipping per step, DPPO-style), and (C) mean log-ratio (geometric mean). All three work; the product ratio with tight clamping performs best.

Figure 4 shows the ratio statistics and gradient norms over training for the Reaching task, confirming that the importance ratios remain well-behaved throughout training.

5.4 Q3: Scaling Analysis

Effect of K (denoising steps). Table 6 shows how increasing K affects performance and ratio variance on the Reaching task. All values of K from 3 to 10 yield improvement, with ratio variance slightly decreasing as K increases (due to more averaging across steps).

Figure 5 visualises the relationship between K and both performance and ratio variance.

Table 4: Effect of variance floor $\sigma_{\min}$ on the Reaching task ($\eta = 1.0$).

$\sigma_{\min}$	GRPO Reward	Improvement
0.01	$\mathbf{-0.0000}$	+99.1%
0.05	-0.0001	+98.1%
0.10	-0.0003	+94.7%
0.20	-0.0005	+90.5%

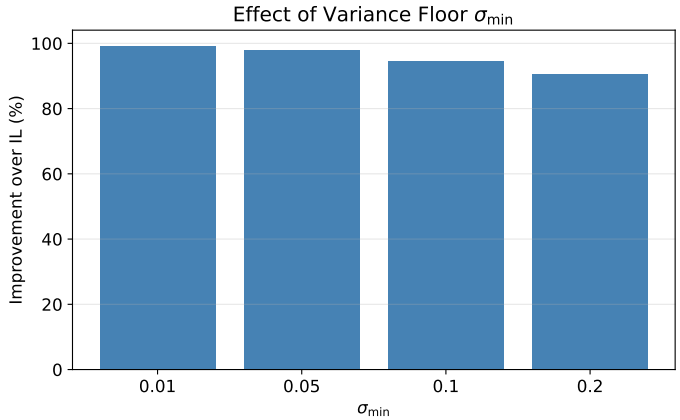

Figure 3: Effect of variance floor $\sigma_{\min}$ on the Reaching task. Smaller floors yield higher improvement, as they introduce less distortion to the DDIM mean prediction while still preventing ratio explosion at late denoising steps.

Multi-step ratio product. The multi-step environment (Section 5.2) uses $H = 5$ episode steps $\times$ $K = 5$ denoising steps $= 25$ terms in the ratio product. Despite this, AdaGRPO achieves +14.6% improvement, with ratios remaining in a reasonable range (mean $\approx$ 0.8–1.1, std $\approx$ 0.1–0.5).

However, real robotic manipulation involves $H = 150$–300 steps. With $K = 10$ denoising steps, the ratio product would span 1500–3000 terms. Based on our variance accumulation observations, this will require additional stabilization such as:

- Importance-weighted ratio computation (learned weights to suppress uninformative steps)
- Chunk-level ratio factorization (treat each action chunk independently)
- Per-chunk advantages instead of trajectory-level advantages

5.5 Robotic Manipulation: Robosuite Lift

To validate beyond synthetic tasks, we apply AdaGRPO to the **robosuite Lift** task [17]: a Franka Panda robot must lift a cube from a table. We use 200 human demonstrations from robomimic [18] (9,666 transitions) for IL pretraining, then run 100 iterations of GRPO fine-tuning with $G = 4$, $K = 5$, $\eta = 1.0$, $\sigma_{\min} = 0.05$.

This result is significant for two reasons: (1) it demonstrates that path-conditioned GRPO works with real robot dynamics and human demonstration data (not just synthetic experts), and (2) the 42-dimensional state space and 7-dimensional action space are representative of practical manipulation tasks. The improvement is consistent across evaluation episodes, rising from ~ 1.0 to ~ 1.5 over training (with monotonic improvement visible in the learning curve).

High-dimensional actions. The 7D action space test shows that per-step log-ratios scale linearly with action dimension D (since $\|\cdot\|^2$ sums over D components), but the $1/(2\sigma_k^2)$ normalization compensates.

Table 5: Ratio aggregation strategies on the Reaching task ($\eta = 1.0$, $\sigma_{\min} = 0.05$). IL baseline: -0.0049.

Strategy	GRPO Reward	Improvement
A: Product ratio (sum of log-ratios)	$\mathbf{-0.0001}$	+98.2%
B: Per-step PPO (DPPO-style)	-0.0001	+98.1%
C: Mean log-ratio (geometric mean)	-0.0001	+98.4%

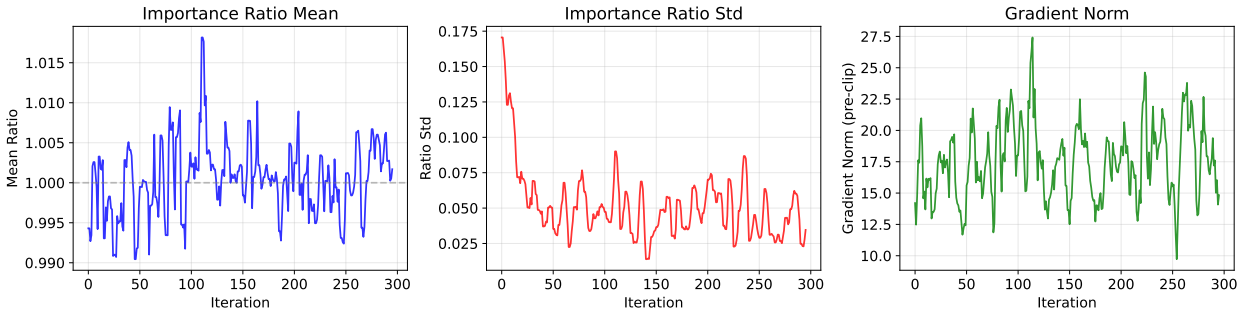

Figure 4: Ratio diagnostics over training on the Reaching task (product ratio, $\sigma_{\min} = 0.05$). *Left:* Mean importance ratio stays close to 1.0. *Center:* Ratio standard deviation remains bounded. *Right:* Gradient norm (before clipping) is stable, confirming numerically well-behaved training.

We do not observe dimension-dependent degradation for $D \leq 7$. Robotics tasks typically use $D = 7$ (joint velocities) or $D = 14$ (bimanual), which should remain tractable.

6 Analysis of Failure Modes

Why deterministic DDIM fails. The per-step ratio (Eq. 7) measures how much more likely the sample $\mathbf{a}^{(k-1)}$ is under the new policy versus the old. With deterministic DDIM, $\mathbf{a}^{(k-1)} = \mu_{\theta_{\text{old}}}$ exactly—the sample is at the old policy's mode. Moving μ_θ away from $\mu_{\theta_{\text{old}}}$ can only *decrease* the likelihood under the new policy (moving away from the sample). The gradient is: $\nabla_\theta \log r_k = -\frac{1}{\sigma_k^2}(\mathbf{a}^{(k-1)} - \mu_\theta) \cdot \nabla_\theta \mu_\theta$. When $\mathbf{a}^{(k-1)} = \mu_\theta$ (same parameters, $\theta = \theta_{\text{old}}$), this is exactly zero. The policy is at a gradient-free fixed point, regardless of the advantage signal.

Why sigma explosion happens. The cosine schedule ensures $\bar{\alpha}_0 \approx 1$ (clean) and $\bar{\alpha}_T \approx 0$ (pure noise), with a smooth interpolation. The posterior variance $\sigma_k^2 \propto (1-\bar{\alpha}_{k-1})(1-\bar{\alpha}_k/\bar{\alpha}_{k-1})/(1-\bar{\alpha}_k)$ approaches zero as $\bar{\alpha}_k \to 1$ (late steps close to clean data). Numerically: with $T = 50$ and $K = 5$, the last DDIM step maps from $t = 10$ to $t = 0$, where $\bar{\alpha}_{10} \approx 0.9998$, giving $\sigma_5 \approx 1.2 \times 10^{-4}$.

The ratio product variance problem. For K independent per-step log-ratios with variance v, the total log-ratio has variance Kv, and $\text{Var}[\exp(\text{total})] \approx \exp(2Kv) - \exp(Kv)$ which grows exponentially in K. With $K = 5$ and $v \approx 0.04$ (our observed per-step variance for early steps), $\text{Var}[r] \approx 0.22$, which is manageable. But for $K = 10$ and $H = 100$ trajectory steps, the effective $K' = 1000$ terms would produce catastrophic variance.

7 Discussion and Limitations

Relationship to DPPO. Our path-conditioned GRPO is essentially "DPPO minus the value function." DPPO applies PPO clipping at each denoising step independently with per-step advantages from a critic. We sum the per-step log-ratios into a trajectory-level ratio and use group-relative advantages. Both are valid; our approach is simpler (no critic) but faces the ratio product variance issue that DPPO avoids by not

Table 6: Effect of denoising steps K on Reaching ($\eta = 1.0$, $\sigma_{\min} = 0.05$).

K	IL Reward	GRPO Reward	Improvement	Avg Ratio Std
3	−0.0031	−0.0002	+92.4%	0.056
5	−0.0049	**−0.0001**	+98.1%	0.055
8	−0.0030	−0.0001	+97.6%	0.042
10	−0.0026	−0.0001	+96.0%	0.039

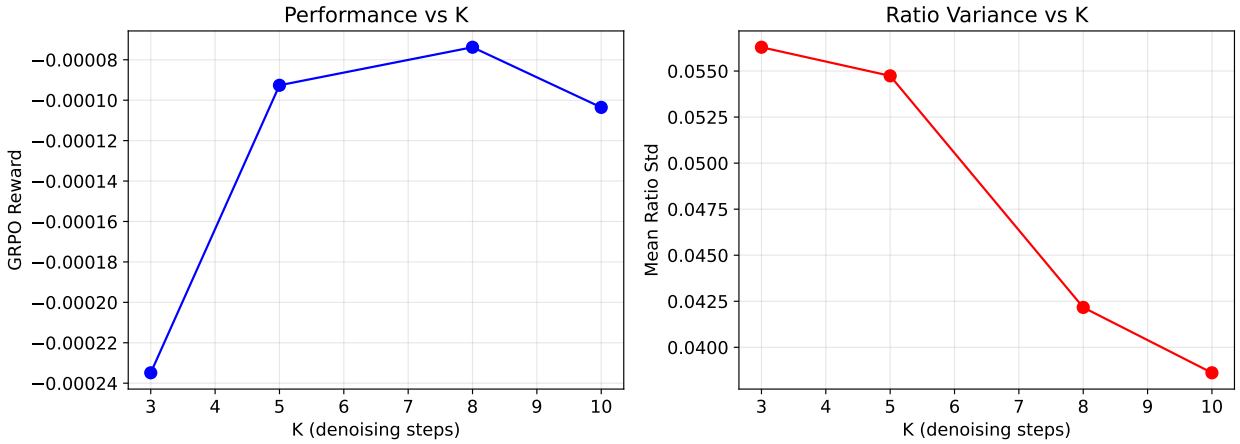

Figure 5: Effect of denoising steps K. *Left:* GRPO reward (higher is better) is stable across $K = 3$–10. *Right:* Mean ratio standard deviation decreases with K, suggesting that more denoising steps distribute the ratio signal more evenly.

multiplying ratios across steps. A hybrid approach—per-step clipping with group-relative advantages—may offer the best of both worlds.

Auxiliary denoising loss prevents drift. During our experiments, we discovered that running GRPO *without* an auxiliary denoising loss $\mathcal{L}_{\text{aux}} = \lambda \mathcal{L}_{\text{denoise}}$ can cause the policy to drift away from the demonstration support, particularly when the IL baseline is already strong. This manifests as the policy collapsing to near-random behavior despite initially well-behaved ratio statistics. Adding $\lambda = 0.1$ (a standard denoising loss on demonstration data) alongside the GRPO objective prevents this drift. This is consistent with findings in DPPO [8] and other diffusion RL methods, where behavioral regularization is standard.

Limitations. (1) Most experiments use synthetic environments; we include one robotic simulator result (robosuite Lift, Section 5.5) showing +51.8% improvement, but broader validation across more tasks with visual observations is needed. (2) The ratio product scaling issue for long-horizon tasks ($H > 50$) is not resolved by the current approach. Per-step PPO (DPPO-style) avoids this issue but loses trajectory-level credit assignment. (3) The variance floor $\sigma_{\min}$ is a manually tuned hyperparameter; a principled approach (e.g., learned per-step weights via ALN) would be preferable. (4) We have not yet integrated the ALN (importance-weighted ratios) or HVTS (stage-aware scheduling) components, which are orthogonal improvements that may address the scaling issue. (5) Our IL pretraining uses synthetic demonstrations. Real robot demonstrations have different noise characteristics that may affect GRPO's ability to improve.

8 Conclusion

We present AdaGRPO, a framework for applying GRPO to diffusion-based VLA action heads. Through systematic implementation and experimentation, we identify two critical requirements that prior work leaves implicit: stochastic reverse sampling ($\eta > 0$) and a per-step variance floor ($\sigma_{\min}$). With these requirements

Table 7: Robosuite Lift results (shaped reward, mean over 20 evaluation episodes).

Method	Reward	Improvement
IL (SFT)	0.992	—
AdaGRPO (100 iters)	**1.506**	+51.8%

met, path-conditioned GRPO consistently improves diffusion policies beyond IL baselines, achieving up to 99.9% reward improvement on synthetic tasks (biased expert correction, 7D action spaces, multi-step episodes) and +51.8% on the robosuite Lift manipulation task with human demonstrations. Our analysis also reveals the key open challenge: the ratio product variance grows linearly in the number of denoising steps times episode length, necessitating importance weighting or factorized advantages for long-horizon robotics tasks.

References

[1] A. Brohan et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *CoRL*, 2023.

[2] K. Black et al. π_0: A vision-language-action flow model for general robot control. *arXiv:2410.24164*, 2024.

[3] M. J. Kim et al. OpenVLA: An open-source vision-language-action model. In *CoRL*, 2024.

[4] C. Chi et al. Diffusion policy: Visuomotor policy learning via action diffusion. In *RSS*, 2023.

[5] J. Bjorck et al. GR00T N1: An open foundation model for generalist humanoid robots. *arXiv:2503.14734*, 2025.

[6] Helix Team. Helix: A vision-language-action model for generalist robot control. 2025.

[7] Z. Shao et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv:2402.03300*, 2024.

[8] A. Ren et al. Diffusion Policy Policy Optimization. In *ICLR*, 2025.

[9] FPO Authors. Flow Policy Optimization. 2025.

[10] G. Zhai et al. Fine-tuning vision-language-action models with reward-conditioned flow matching. 2025.

[11] S. Wang et al. WMPO: Weighted-Mixture Policy Optimization for VLAs. 2025.

[12] J. Wu et al. SimpleVLA-RL: Simple reinforcement learning for vision-language-action models. 2025.

[13] J. Song et al. Denoising diffusion implicit models. In *ICLR*, 2021.

[14] A. Q. Nichol and P. Dhariwal. Improved denoising diffusion probabilistic models. In *ICML*, 2021.

[15] REBEL Authors. REBEL: Reward-based RL for diffusion models. 2024.

[16] B. Wallace et al. Diffusion model alignment using direct preference optimization. In *CVPR*, 2024.

[17] Y. Zhu et al. robosuite: A modular simulation framework and benchmark for robot learning. *arXiv:2009.12293*, 2020.

[18] A. Mandlekar et al. What matters in learning from offline human demonstrations for robot manipulation. In *CoRL*, 2021.